

Expense Tracker: JDBC Console App

Assumptions: MySQL, Connector/J on the classpath, database named expense_db. Change the URL, username and password to match your setup.

Q1: SpendingCategory table and sample data

```
CREATE DATABASE expense_db;
USE expense_db;

CREATE TABLE SpendingCategory (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL
);

INSERT INTO SpendingCategory (name) VALUES
    ('Food'),
    ('Travel'),
    ('Shopping'),
    ('Bills'),
    ('Entertainment');

SELECT * FROM SpendingCategory;
```

Output:

```
+---+-----+
| id | name      |
+---+-----+
|  1 | Food      |
|  2 | Travel    |
|  3 | Shopping  |
|  4 | Bills     |
|  5 | Entertainment |
+---+-----+
```

Supporting tables (needed for Q3 to Q5, since expenses belong to a logged-in user):

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL
);

CREATE TABLE expenses (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    expense_date DATE NOT NULL,
    category_id INT NOT NULL,
    description VARCHAR(200),
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (category_id) REFERENCES SpendingCategory(id)
);

INSERT INTO users (username, password) VALUES ('student1', 'pass123');
```

Shared helper: DBUtil.java

```
// DBUtil.java
// Helper: provides JDBC connections for the whole application.

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBUtil {
    private static final String URL = "jdbc:mysql://localhost:3306/expense_db";
    private static final String USER = "root";
    private static final String PASS = "root123";
```

```

        public static Connection getConnection() throws SQLException {
            return DriverManager.getConnection(URL, USER, PASS);
        }
    }
}

```

Q2: Expense class

```

// Expense.java
// Model (JavaBean) representing one expense. The id is generated by the database,
// so the constructor initializes every field except id.

import java.time.LocalDate;

public class Expense {
    private int id;
    private int userId;
    private double amount;
    private LocalDate date;
    private int categoryId;
    private String description;

    public Expense(int userId, double amount, LocalDate date, int categoryId, String description) {
        this.userId = userId;
        this.amount = amount;
        this.date = date;
        this.categoryId = categoryId;
        this.description = description;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getUserId() { return userId; }
    public void setUserId(int userId) { this.userId = userId; }

    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }

    public LocalDate getDate() { return date; }
    public void setDate(LocalDate date) { this.date = date; }

    public int getCategoryId() { return categoryId; }
    public void setCategoryId(int categoryId) { this.categoryId = categoryId; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
}

```

Q3 and Q4: ExpenseController (addExpense and monthly report)

addExpense () shows the categories, asks for the amount, date, category and description, and inserts the expense for the logged-in user. The monthly report uses SUM () and GROUP BY.

```

// ExpenseController.java
// Controller: handles console input and JDBC operations for expenses.

import java.sql.*;
import java.time.LocalDate;
import java.time.YearMonth;
import java.time.format.DateTimeParseException;
import java.util.Scanner;

public class ExpenseController {

    // ----- Q3: add a new expense for the logged-in user -----
    public void addExpense(int userId, Scanner sc) {
        showCategories();

        try {
            System.out.print("Enter amount: ");
            double amount = Double.parseDouble(sc.nextLine().trim());
            if (amount <= 0) {
                System.out.println("Amount must be greater than 0.");
            }
        }
    }
}

```

```

        return;
    }

    System.out.print("Enter date (yyyy-MM-dd): ");
    LocalDate date = LocalDate.parse(sc.nextLine().trim());

    System.out.print("Enter category ID: ");
    int categoryId = Integer.parseInt(sc.nextLine().trim());

    System.out.print("Enter description: ");
    String description = sc.nextLine().trim();

    Expense expense = new Expense(userId, amount, date, categoryId, description);
    String sql = "INSERT INTO expenses (user_id, amount, expense_date, category_id, description) "
        + "VALUES (?, ?, ?, ?, ?)";

    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, expense.getUserId());
        ps.setDouble(2, expense.getAmount());
        ps.setDate(3, java.sql.Date.valueOf(expense.getDate()));
        ps.setInt(4, expense.getCategoryId());
        ps.setString(5, expense.getDescription());

        if (ps.executeUpdate() > 0) {
            System.out.println("Expense added successfully.");
        }
    } catch (NumberFormatException e) {
        System.out.println("Invalid number entered. Please try again.");
    } catch (DateTimeParseException e) {
        System.out.println("Invalid date. Use the format yyyy-MM-dd (example: 2026-09-24).");
    } catch (SQLIntegrityConstraintViolationException e) {
        System.out.println("Invalid category ID. Please choose one from the list.");
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}

// Helper: prints the available categories
private void showCategories() {
    try (Connection con = DBUtil.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT id, name FROM SpendingCategory")) {

        System.out.println("\nCategories:");
        while (rs.next()) {
            System.out.println("  " + rs.getInt("id") + ". " + rs.getString("name"));
        }
    } catch (SQLException e) {
        System.out.println("Could not load categories: " + e.getMessage());
    }
}

// ----- Q4: monthly report grouped by category -----
public void showMonthlyReport(int userId, Scanner sc) {
    System.out.print("Enter month (yyyy-MM): ");
    YearMonth ym;
    try {
        ym = YearMonth.parse(sc.nextLine().trim());
    } catch (DateTimeParseException e) {
        System.out.println("Invalid month. Use the format yyyy-MM (example: 2026-09).");
        return;
    }

    String sql = "SELECT c.name, SUM(e.amount) AS total "
        + "FROM expenses e JOIN SpendingCategory c ON e.category_id = c.id "
        + "WHERE e.user_id = ? AND YEAR(e.expense_date) = ? AND MONTH(e.expense_date) = ? "
        + "GROUP BY c.name "
        + "ORDER BY total DESC";

    try (Connection con = DBUtil.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

```

```

ps.setInt(1, userId);
ps.setInt(2, ym.getYear());
ps.setInt(3, ym.getMonthValue());

try (ResultSet rs = ps.executeQuery()) {
    System.out.println("\nMonthly Report for " + ym);
    System.out.println("+-----+");
    System.out.printf("| %-20s | %12s |%n", "Category", "Total (Rs.)");
    System.out.println("+-----+");

    double grandTotal = 0;
    boolean found = false;
    while (rs.next()) {
        found = true;
        double total = rs.getDouble("total");
        grandTotal += total;
        System.out.printf("| %-20s | %12.2f |%n", rs.getString("name"), total);
    }

    if (!found) {
        System.out.printf("| %-35s |%n", "No expenses found for this month");
    }
    System.out.println("+-----+");
    System.out.printf("| %-20s | %12.2f |%n", "TOTAL", grandTotal);
    System.out.println("+-----+");
}
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
}

```

Sample output of addExpense():

Categories:

1. Food
2. Travel
3. Shopping
4. Bills
5. Entertainment

Enter amount: 250
Enter date (yyyy-MM-dd): 2026-09-20
Enter category ID: 1
Enter description: Lunch with friends
Expense added successfully.

Sample output of the monthly report:

Monthly Report for 2026-09

Category	Total (Rs.)
Food	1850.00
Bills	1200.00
Travel	600.00
Entertainment	450.00
TOTAL	4100.00

Key SQL idea: SUM(e.amount) adds up the amounts, and GROUP BY c.name makes one result row per category.

Q5: AI budgeting tips after login

I cannot run ChatGPT for you, and your teacher will expect your own screenshot. Do it yourself like this:

Step 1: Paste this prompt into ChatGPT (or another AI tool):

"Give me 3 short, personalized budgeting tips for college students in India who spend mostly on food, travel and entertainment."

Step 2: Take a screenshot of the prompt and the response.

Step 3: Copy the 3 tips into the app below (replace the sample tips with the ones you receive). This follows the hint: if the API is not available, manually copy-paste the tips.

Sample tips (format only; use your own ChatGPT output in your submission):

- **Plan food first.** Set a weekly food limit and cook or eat at the mess more often. Food is usually a student's biggest flexible expense.
- **Use student discounts.** Use student passes for travel, and student plans for streaming and software, instead of paying full price.
- **Pay yourself first.** Move a small fixed amount (even Rs. 200 to Rs. 500) into savings as soon as money arrives, then spend the rest.

ExpenseApp.java (login, AI Tips section and menu):

```
// ExpenseApp.java
// Entry point: logs the user in, shows the AI Tips section, then displays the menu.

import java.sql.*;
import java.util.Scanner;

public class ExpenseApp {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ExpenseController controller = new ExpenseController();

        System.out.print("Username: ");
        String username = sc.nextLine().trim();
        System.out.print("Password: ");
        String password = sc.nextLine().trim();

        int userId = login(username, password);
        if (userId == -1) {
            System.out.println("Invalid username or password.");
            return;
        }
        System.out.println("\nWelcome, " + username + "!");

        showAiTips(); // AI Tips section shown right after login

        while (true) {
            System.out.println("\n1. Add expense");
            System.out.println("2. Monthly report");
            System.out.println("3. Exit");
            System.out.print("Choose an option: ");
            String choice = sc.nextLine().trim();

            if (choice.equals("1")) {
                controller.addExpense(userId, sc);
            } else if (choice.equals("2")) {
                controller.showMonthlyReport(userId, sc);
            } else if (choice.equals("3")) {
                System.out.println("Goodbye!");
                break;
            } else {
                System.out.println("Invalid option.");
            }
        }
    }

    // Returns the user's id if the login is correct, otherwise -1
    private static int login(String username, String password) {
        String sql = "SELECT id FROM users WHERE username = ? AND password = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) return rs.getInt("id");
            }
        }
    }
}
```

```

    }
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
return -1;
}

// Tips generated with ChatGPT and copied in manually
private static void showAiTips() {
    System.out.println("===== AI Tips =====");
    System.out.println("1. Plan food first: set a weekly food limit and cook or eat at the mess more often.");
    System.out.println("2. Use student discounts for travel, streaming and software.");
    System.out.println("3. Pay yourself first: move Rs. 200-500 into savings as soon as money arrives.");
    System.out.println("=====");
}
}

```

Sample output after login:

```

Welcome, student1!
===== AI Tips =====
1. Plan food first: set a weekly food limit and cook or eat at the mess more often.
2. Use student discounts for travel, streaming and software.
3. Pay yourself first: move Rs. 200-500 into savings as soon as money arrives.
=====

1. Add expense
2. Monthly report
3. Exit
Choose an option:

```

Note: passwords are stored as plain text here for simplicity. A real app should hash them (for example with BCrypt).